

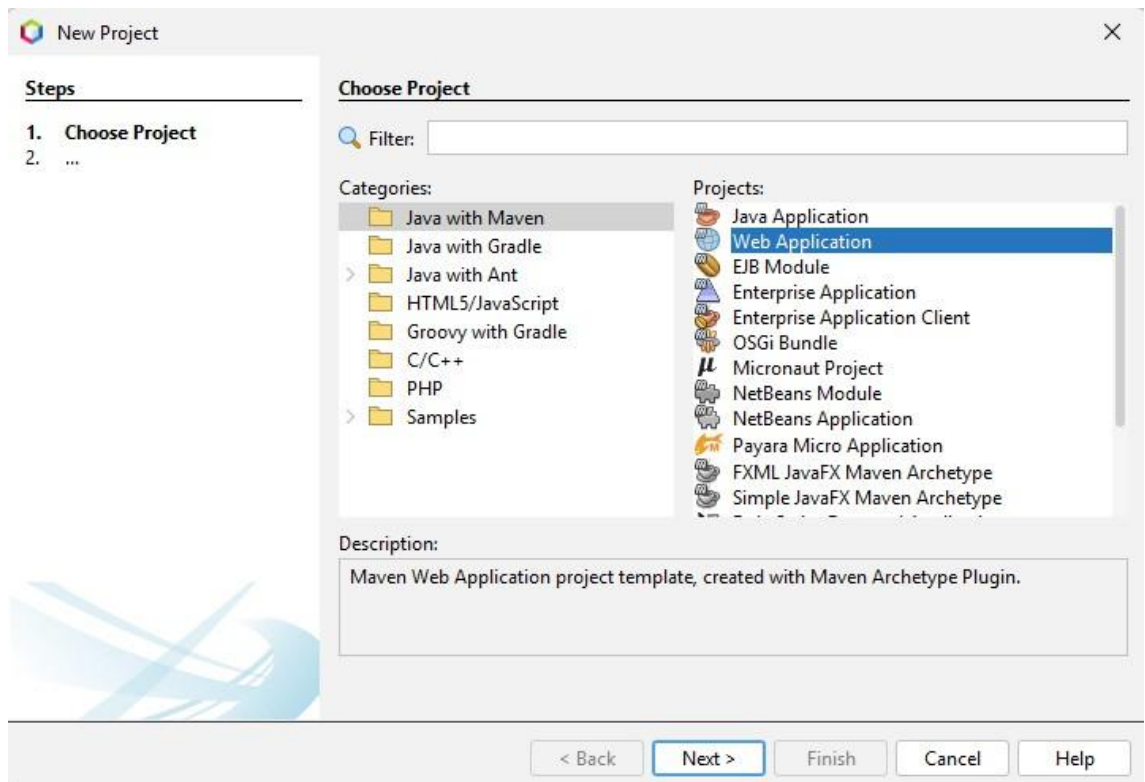
SISTEMA SEGURO CONTRA DESBORDAMIENTO (OVERFLOW) Manual

Paso 1: Configuración del Entorno y Creación del Proyecto

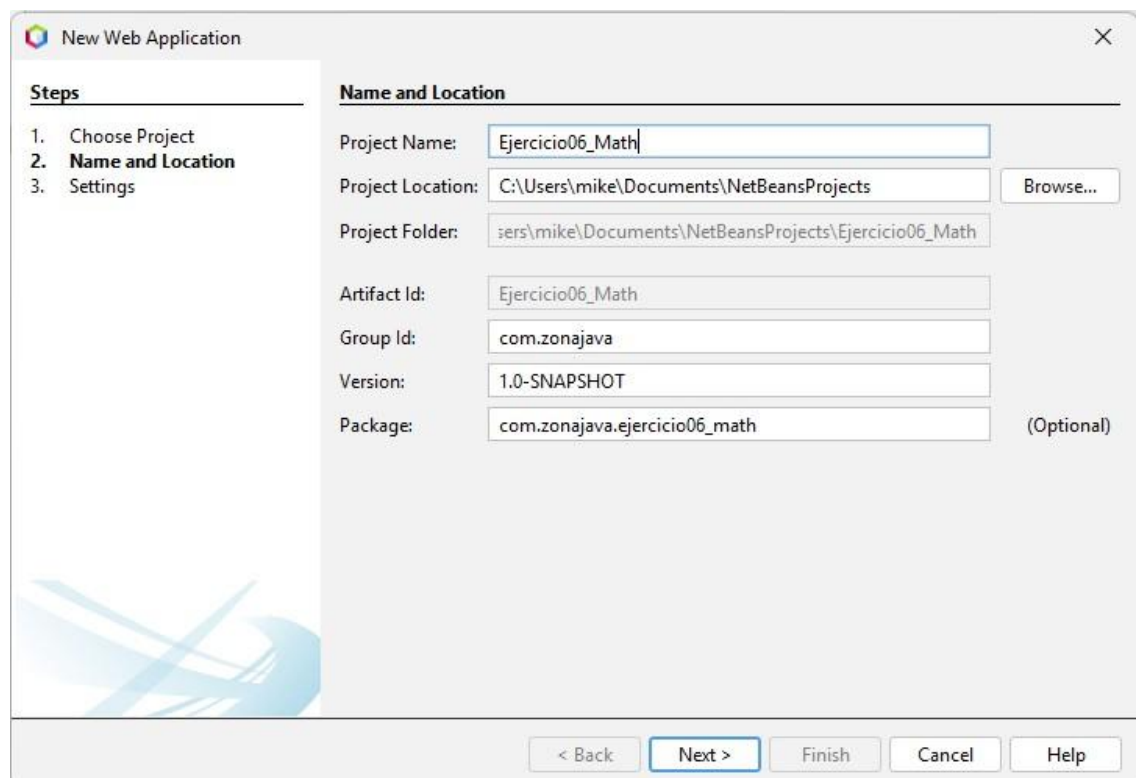
1. Crea un nuevo proyecto: File > New Project.



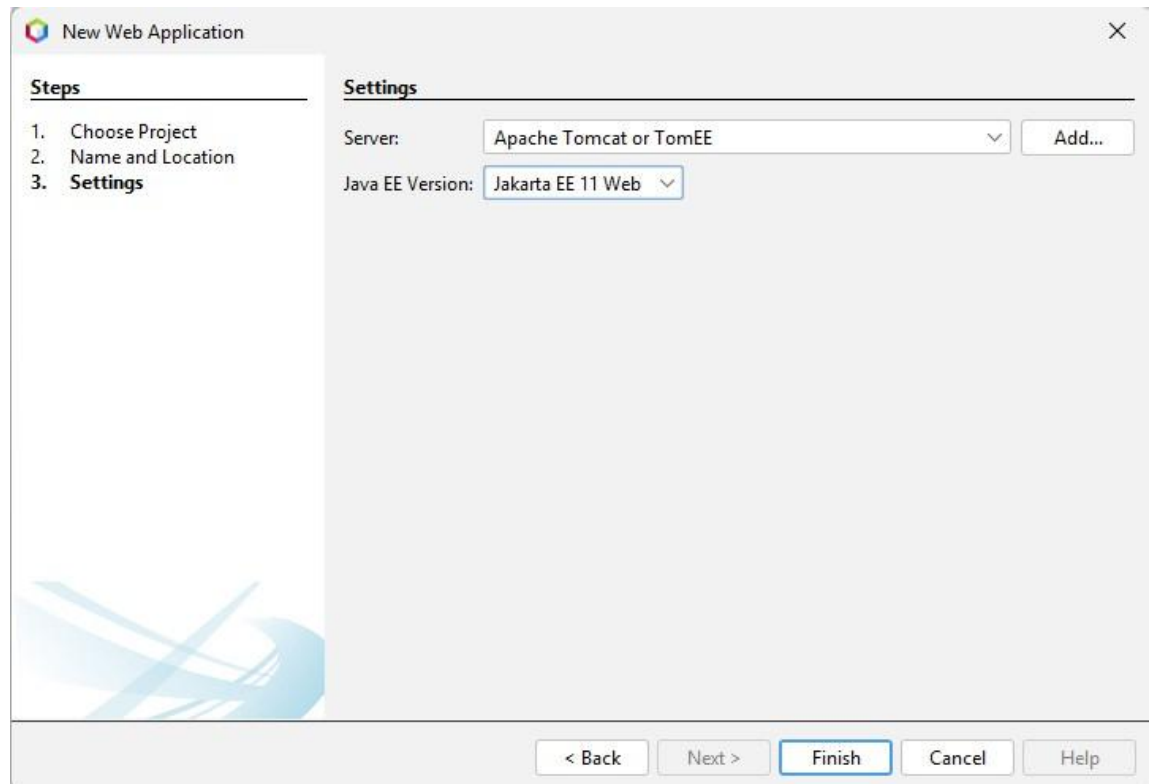
2. Selecciona Java with Maven > Web Application.



3. Nombra el proyecto como Ejercicio06_Math.

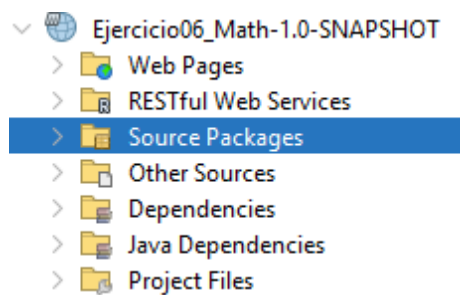


4. Asegúrate de seleccionar una versión de Jakarta EE (el proyecto indica compatibilidad con Jakarta EE 11 en sus recursos).



Paso 2: Organización de los Paquetes de Código Fuente

Dentro de la carpeta Source Packages, debes crear la siguiente jerarquía de paquetes para mantener el orden (MVC):



1. Crear un paquete base (com.mycompany.Ejercicio06_Math.controlador).

New Java Package

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Package Name: com.mycompany.Ejercicio06_Math.controlador

Project: Ejercicio06_Math-1.0-SNAPSHOT

Location: Source Packages

Created Folder: cio06_Math\src\main\java\com\mycompany\Ejercicio06_Math\controlador

< Back Next > Finish Cancel Help

2. Crear un paquete base (com.mycompany.Ejercicio06_Math.modelo).

New Java Package

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Package Name: com.mycompany.Ejercicio06_Math.modelo

Project: Ejercicio06_Math-1.0-SNAPSHOT

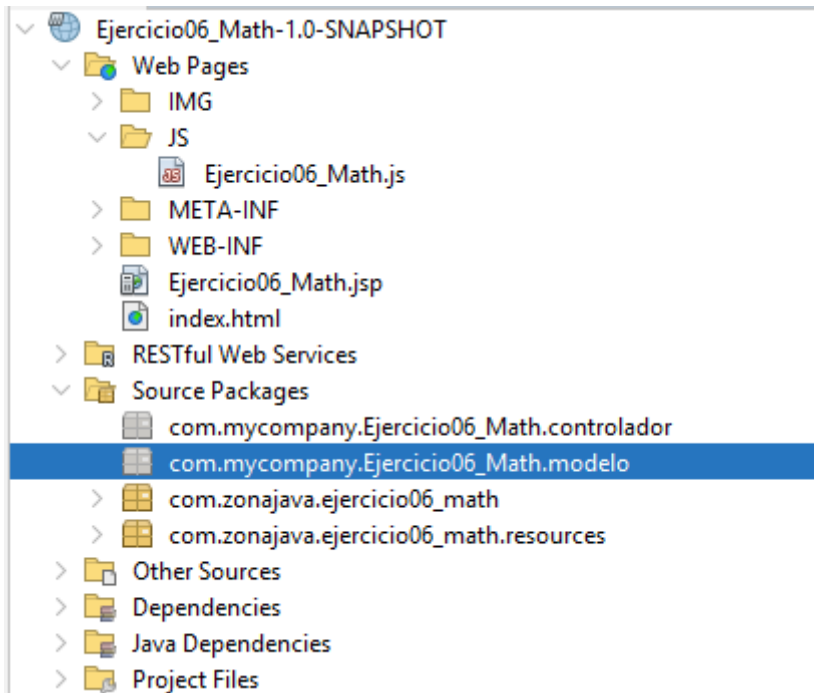
Location: Source Packages

Created Folder: ercicio06_Math\src\main\java\com\mycompany\Ejercicio06_Math\modelo

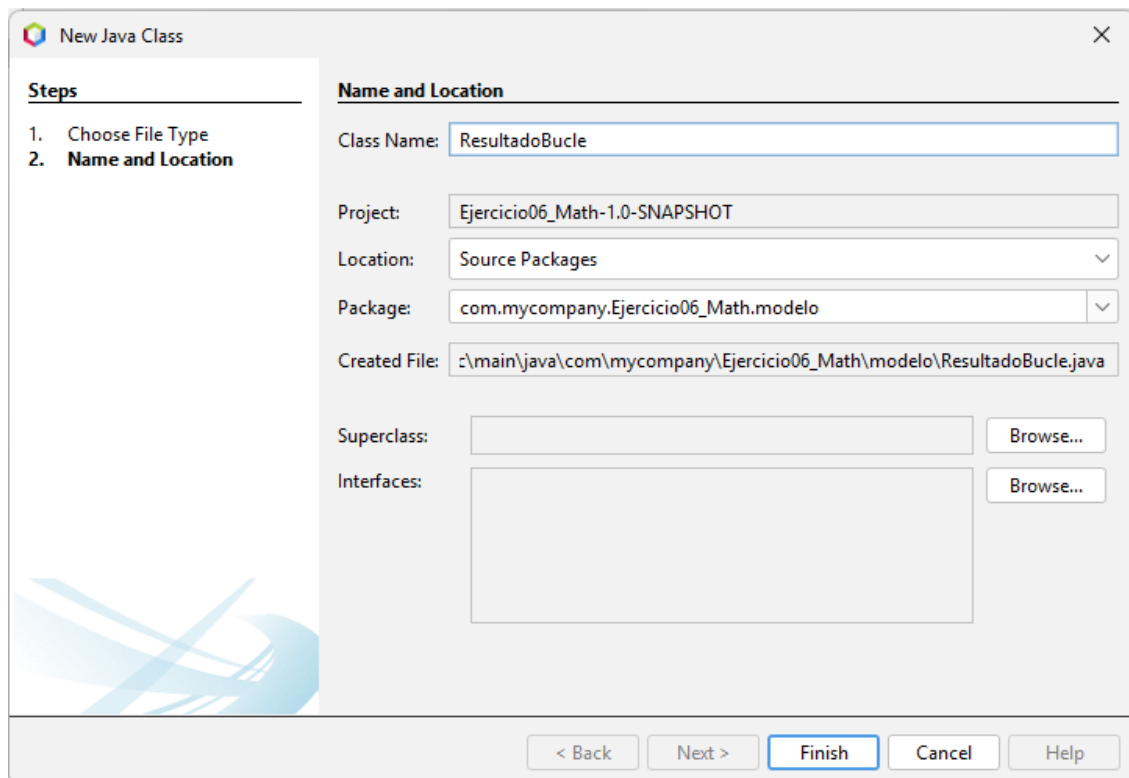
< Back Next > Finish Cancel Help

Paso 3: Desarrollo de la Capa de Datos (Modelo)

Crea las clases Java básicas que representarán tus datos en el paquete `.modelo`:



- ResultadoBucle.java



CODIGO:

```
package com.mycompany.Ejercicio06_Math.modelo;

import java.util.ArrayList;
import java.util.List;

public class ResultadoBucle {
    private int numeroBase;
    private List<String> pasos;
    private boolean huboOverflow;
    private String mensajeAdvertencia;

    // Constructor por defecto
    public ResultadoBucle() {
        this.pasos = new ArrayList<>();
    }

    public ResultadoBucle(int numeroBase) {
        this.numeroBase = numeroBase;
        this.pasos = new ArrayList<>();
        this.huboOverflow = false;
        this.mensajeAdvertencia = "";
    }

    // Métodos de utilidad para ir llenando los datos
    public void agregarPaso(int exponente, int resultado) {
        this.pasos.add("[Paso " + exponente + "]: " + numeroBase + "^" +
exponente + " = " + String.format("%,d", resultado));
    }

    // Getters y Setters
    public int getNumeroBase() {
        return numeroBase;
    }

    public void setNumeroBase(int numeroBase) {
        this.numeroBase = numeroBase;
    }

    public List<String> getPasos() {
        return pasos;
    }

    public void setPasos(List<String> pasos) {
        this.pasos = pasos;
    }

    public boolean isHuboOverflow() {
```

```

        return huboOverflow;
    }

    public void setHuboOverflow(boolean huboOverflow) {
        this.huboOverflow = huboOverflow;
    }

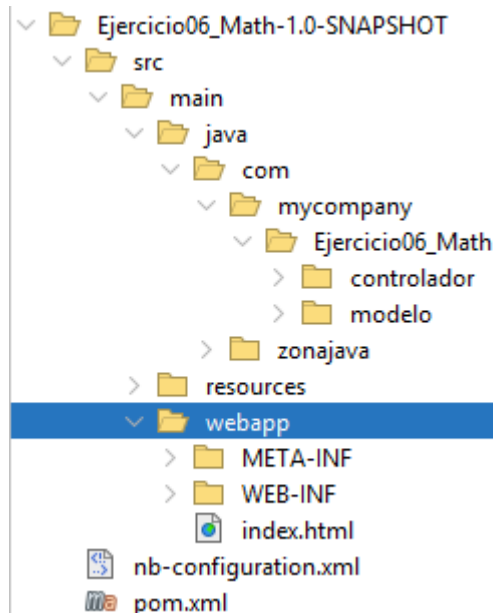
    public String getMensajeAdvertencia() {
        return mensajeAdvertencia;
    }

    public void setMensajeAdvertencia(String mensajeAdvertencia) {
        this.mensajeAdvertencia = mensajeAdvertencia;
    }
}

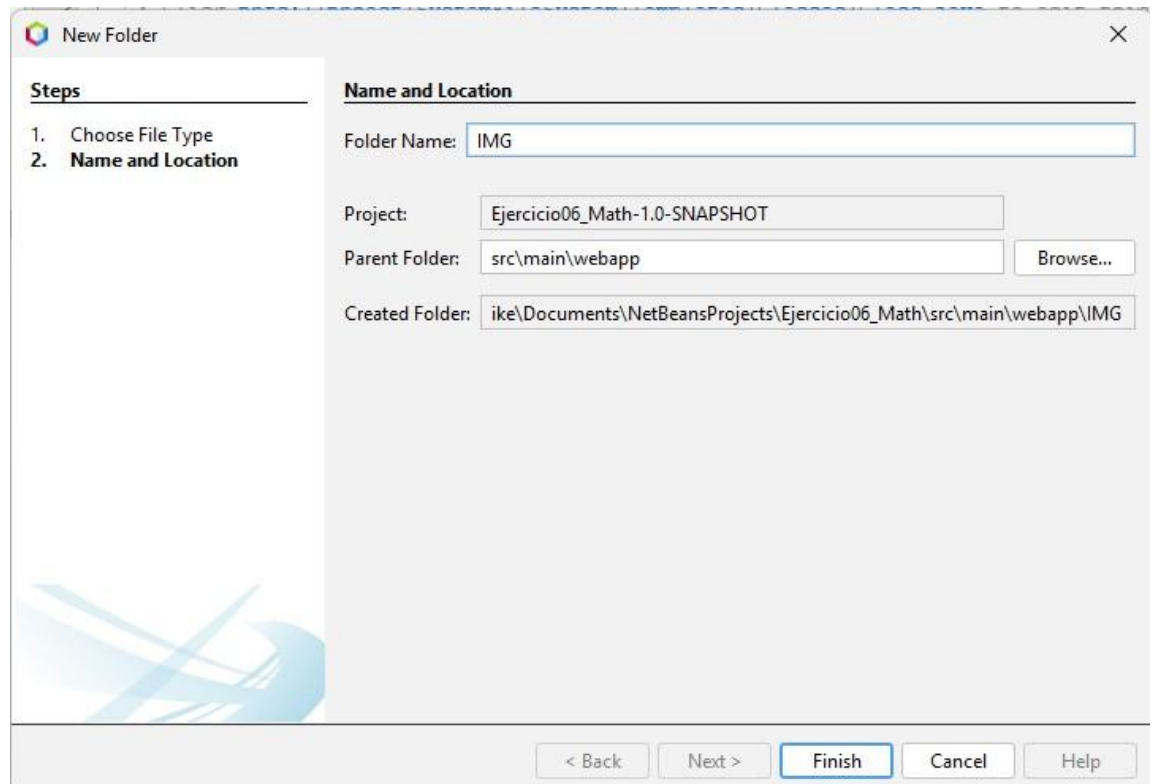
```

Paso 4: Creación de la Interfaz de Usuario (Vista)

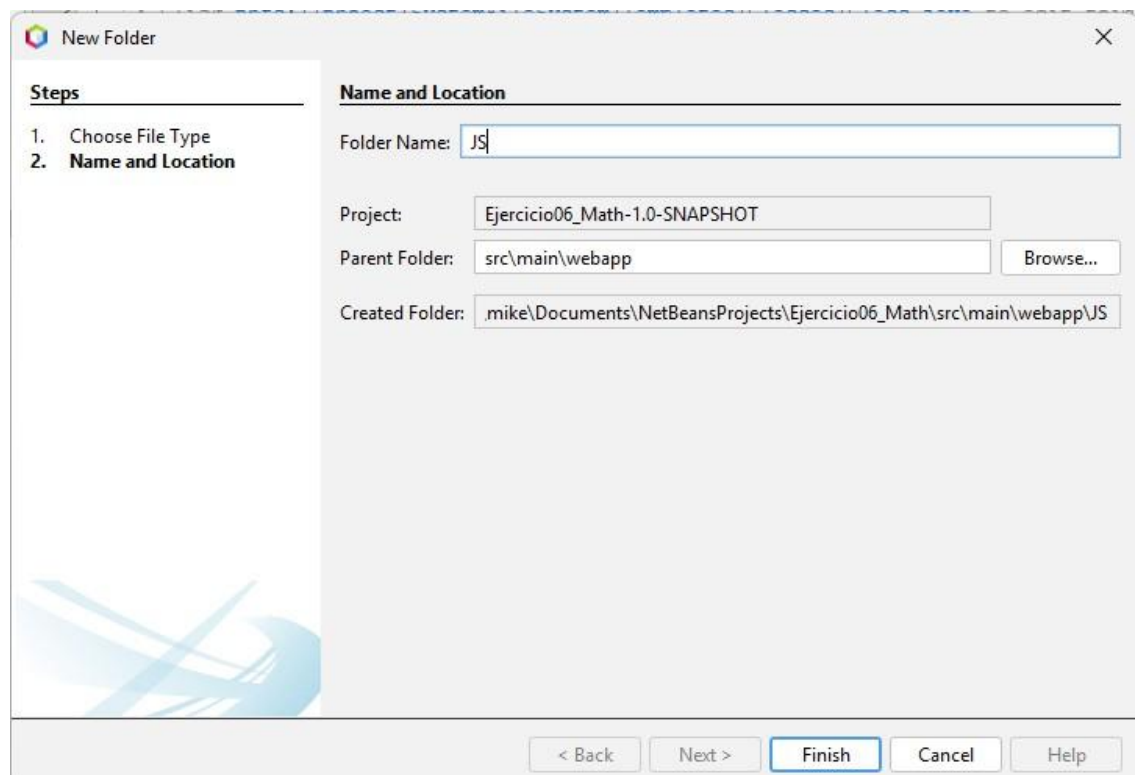
En la carpeta Web Pages, organiza tus archivos front-end:



1. Imágenes: Crea una carpeta IMG y añade los logotipos (LOGO IESTP.png, logo DPW.png).



2. Scripts: Crea una carpeta JS y crea los archivos de lógica del cliente: Ejercicio06_Math.js.



New JavaScript File

Steps

1. Choose File Type
2. Name and Location

Name and Location

JavaScript File Name:

Project:

Location:

Folder:

Created File:

CODIGO:

```
function calcularPotencias() {

    const INT32_MAX = 2147483647;
    const base =
parseInt(document.getElementById('baseNumber').value);

    const consoleLog = document.getElementById('consoleLog');
    const overflowAlert =
document.getElementById('overflowAlert');
    const alertMessage = document.getElementById('alertMessage');

    consoleLog.innerHTML = "";
    overflowAlert.classList.add('hidden');

    if (isNaN(base) || base < 2) {
        consoleLog.innerHTML =
            `
                ✖ Por favor introduce un número entero válido
mayor o igual a 2.
            `;
        return;
    }

    let resultadoActual = base;
    let exponente = 1;

    consoleLog.innerHTML +=
        `

Iniciando bucle de potencias para la base ${base}

`;

    consoleLog.innerHTML +=
```



```

        `<div>
            [Paso ${exponente}] :
            ${base}<sup>${exponente}</sup> =
            ${resultadoActual.toLocaleString()}
        </div>`;

while (true) {

    if (resultadoActual > INT32_MAX / base) {

        let valorDesbordado = resultadoActual * base;

        alertMessage.innerHTML =
            `La operación
            <strong>${resultadoActual.toLocaleString()} ×
${base}</strong>
            produciría
            <strong>${valorDesbordado.toLocaleString()}</stro
ng>,
            superando el límite máximo de un entero de 32
bits.`;

        overflowAlert.classList.remove('hidden');

        consoleLog.innerHTML +=
            `<div class="text-amber-500 font-bold mt-2">
                ⚠ Bucle detenido para evitar Overflow.
            </div>`;

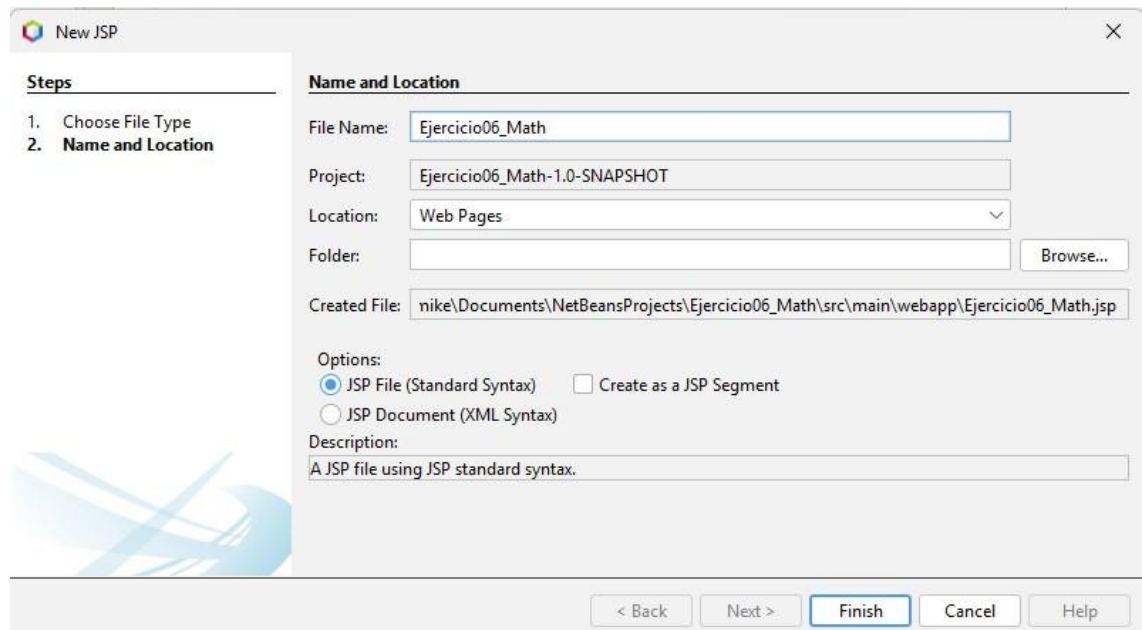
        break;
    }

    resultadoActual *= base;
    exponente++;

    consoleLog.innerHTML +=
        `<div>
            [Paso ${exponente}] :
            ${base}<sup>${exponente}</sup> =
            ${resultadoActual.toLocaleString()}
        </div>`;
    }
}

```

3. Páginas JSP: Crea en la raíz de Web Pages los archivos: Ejercicio06_Math.jsp.



CODIGO:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejercicio 6 - Overflow</title>

  <script
src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
</head>

<body class="bg-slate-900 text-slate-100 min-h-screen">

  <!-- ENCABEZADO -->
<!-- ENCABEZADO -->
<header>

  <!-- Barra Superior -->
  <div class="bg-white shadow-lg">
    <div class="max-w-7xl mx-auto px-8 py-5">
      <h1 class="text-4xl font-extrabold text-blue-500 uppercase
text-center">
        EJERCICIO 6 MATH
      </h1>
    </div>
  </div>
</div>
```

```

<!-- Banner -->
<div class="border-b border-slate-700 bg-slate-800">

    <div class="max-w-7xl mx-auto px-6 py-8">

        <div class="grid md:grid-cols-3 gap-6 items-center">

            <!-- Logo Izquierdo -->
            <div class="flex justify-center md:justify-start">

                <div class="bg-white p-3 rounded-2xl shadow-xl">
                    
                </div>

            </div>

            <!-- Título Central -->
            <div class="text-center">

                <h2 class="text-3xl font-bold text-white">
                    Sistema Seguro contra Overflow
                </h2>

                <p class="text-slate-300 mt-2">
                    Multiplicación sucesiva con control de enteros de
32 bits
                </p>

            </div>

            <!-- Logo Derecho -->
            <div class="flex justify-center md:justify-end">

                <div class="bg-white p-3 rounded-2xl shadow-xl">
                    
                </div>

            </div>

        </div>

    </div>

```

```

        </div>

</div>

</header>
<!-- CONTENIDO -->
<main class="max-w-5xl mx-auto px-4 py-10">

    <div class="bg-slate-800 p-8 rounded-3xl shadow-2xl border
border-slate-700">

        <!-- Título -->
        <div class="mb-6">

            <span class="inline-block text-xs font-semibold uppercase
tracking-wider text-indigo-300 bg-indigo-950 px-4 py-2 rounded-full
border border-indigo-700">
                Ejercicio 6_Math
            </span>

            <h2 class="text-3xl font-bold mt-4">
                Sistema Seguro contra Desbordamiento (Overflow)
            </h2>

        </div>

        <!-- Descripción -->
        <div class="bg-slate-900 border-l-4 border-indigo-500 p-5
rounded-r-xl text-slate-300">

            <p class="text-indigo-400 font-semibold uppercase text-xs
mb-2">
                Enunciado del ejercicio
            </p>

            <p>
                Escribe un programa que intente multiplicar de forma
sucesiva
                un número entero por sí mismo. Utiliza el control de
aritmética
                exacta para detener el bucle limpiamente desplegando
un mensaje
                de advertencia cuando la operación ya no pueda
almacenarse
                en un entero de 32 bits.
            </p>

        </div>

```

```

<!-- Entrada -->
<div class="mt-8">

    <label
        for="baseNumber"
        class="block mb-3 text-sm font-medium text-slate-
300">

        Introduce un número entero:

    </label>

    <div class="flex flex-col md:flex-row gap-4">

        <input
            type="number"
            id="baseNumber"
            value="2"
            min="2"
            placeholder="Ejemplo: 2, 3, 5..."
            class="flex-1 bg-slate-950 border border-slate-
600 rounded-xl px-4 py-3 text-white focus:outline-none focus:border-
indigo-500"

            >

        <button
            onclick="calcularPotencias()"
            class="bg-indigo-600 hover:bg-indigo-500
transition px-6 py-3 rounded-xl font-semibold shadow-lg">

            Iniciar Bucle

        </button>

    </div>

</div>

<!-- Alerta -->
<div
    id="overflowAlert"
    class="hidden mt-6 bg-amber-950 border border-amber-500
p-4 rounded-xl">

    <h3 class="text-amber-400 font-bold">
        ⚠ Advertencia de Overflow
    </h3>

```

```

        <p
          id="alertMessage"
          class="text-amber-200 mt-2 text-sm">
        </p>

      </div>

      <!-- Consola -->
      <div class="mt-8">

        <h3 class="uppercase text-xs tracking-wider text-slate-
400 mb-3">
          Registro de operaciones
        </h3>

        <div
          id="consoleLog"
          class="bg-slate-950 border border-slate-700 rounded-
xl p-4 text-emerald-400 font-mono text-sm max-h-80 overflow-y-auto">

          <span class="text-slate-500 italic">
            Esperando datos...
          </span>

        </div>

      </div>

      <!-- Pie -->
      <div class="mt-6 flex justify-between text-xs text-slate-
500">

        <span>Control de flujo preventivo activado</span>

        <span>INT32 MAX = 2,147,483,647</span>

      </div>

    </div>

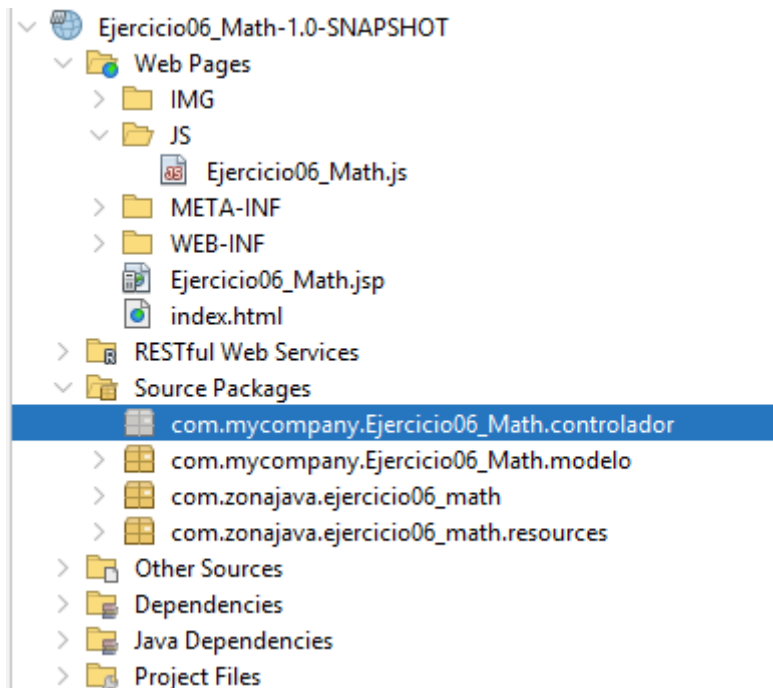
  </main>

<styles src="JS/Ejercicio06_Math"></styles>

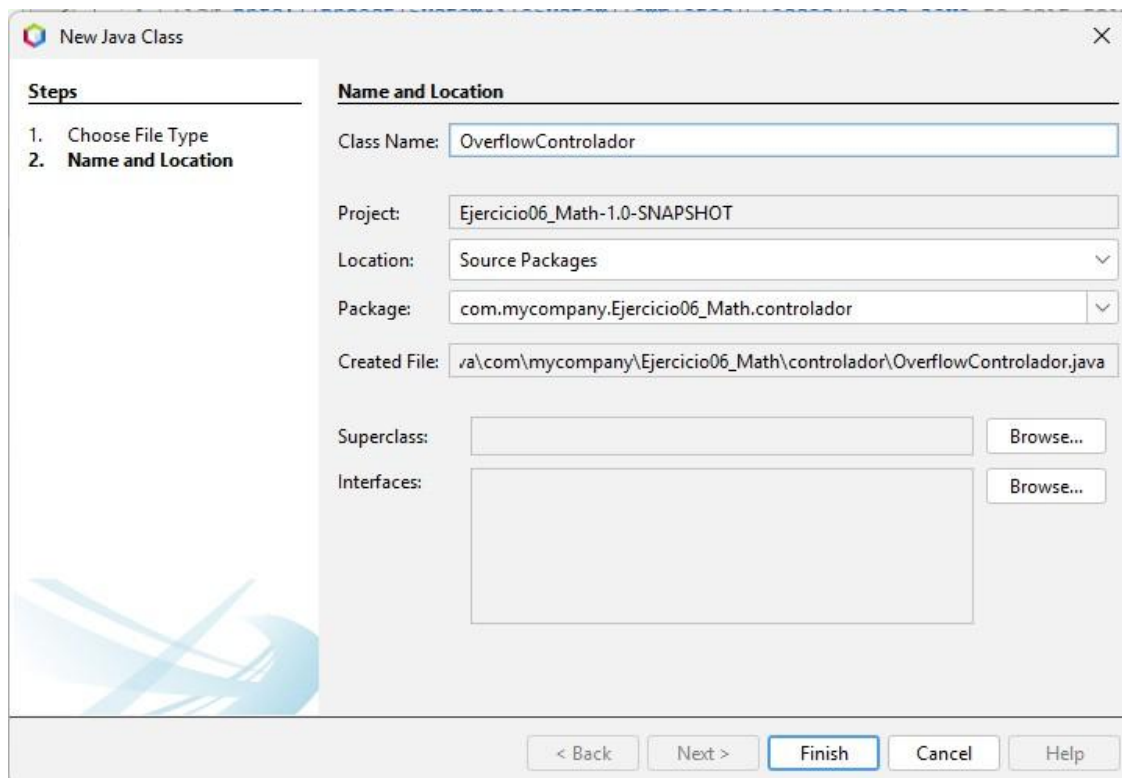
```

Paso 5: Implementación de la Lógica (Controlador)

Crea los Servlets en el paquete `.controlador` para manejar las peticiones de los formularios JSP:



- `OverflowControlador.java`



CODIGO:

```
package com.mycompany.Ejercicio06_Math.controlador;

import com.mycompany.Ejercicio06_Math.modelo.ResultadoBucle;

public class OverflowControlador {

    /**
     * Ejecuta el bucle de potencias sucesivas de forma segura
     controlando
     * la aritmética exacta para evitar desbordamientos del tipo int.
     * * @param base Número entero enviado desde la vista
     * @return Objeto ResultadoBucle con el registro de pasos y
     advertencias
     */
    public ResultadoBucle procesarBuclePotencias(int base) {
        ResultadoBucle resultado = new ResultadoBucle(base);

        if (base < 2) {
            resultado.setHuboOverflow(false);
            resultado.setMensajeAdvertencia("Por favor, introduce un
número entero mayor o igual a 2.");
            return resultado;
        }

        int resultadoActual = base;
        int exponente = 1;

        // Registrar el paso inicial de forma segura
        resultado.agregarPaso(exponente, resultadoActual);

        // Bucle de multiplicación sucesiva
        while (true) {
            /* * CONTROL DE ARITMÉTICA EXACTA PREVENTIVA:
             * Comprobamos si la multiplicación 'resultadoActual * base'
             superará
             * el valor máximo de un int (Integer.MAX_VALUE =
             2,147,483,647).
             */
            if (resultadoActual > Integer.MAX_VALUE / base) {
                // El bucle se detiene limpiamente de forma segura
                resultado.setHuboOverflow(true);

                // Calculamos el valor teórico usando un long (64 bits)
                para mostrar el desborde real en la advertencia
                long valorDesbordadoTeorico = (long) resultadoActual *
base;
            }
        }
    }
}
```



```

        String advertencia = "¡Advertencia de Overflow detectada!
La operación ("
        + String.format("%,d", resultadoActual) + " * " +
base
        + ") daría como resultado " +
String.format("%,d", valorDesbordadoTeorico)
        + ", lo cual ya no puede almacenarse en un tipo
de dato 'int' de 32 bits.";

        resultado.setMensajeAdvertencia(advertencia);
        break;
    }

    // Operación segura garantizada por el control previo
    resultadoActual = resultadoActual * base;
    exponente++;
    resultado.agregarPaso(exponente, resultadoActual);
}

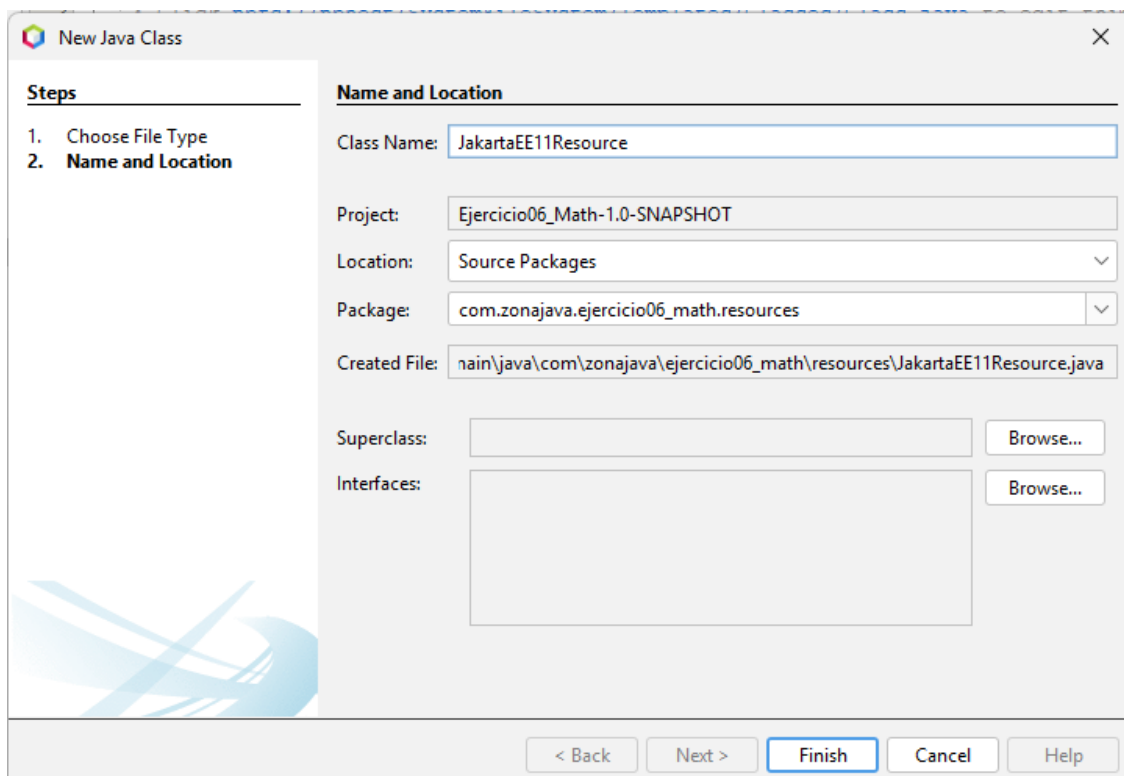
return resultado;
}
}

```

Paso 6: Configuración de Servicios REST

Para habilitar servicios web modernos en tu aplicación:

1. Crea la clase JakartaRestConfiguration.java en el paquete principal. Esta clase suele extender de Application para definir la ruta base de los servicios.



CODIGO:

```
package com.zonajava.ejercicio06_math.resources;

import com.mycompany.Ejercicio06_Math.controlador.OverflowControlador;
import com.mycompany.Ejercicio06_Math.modelo.ResultadoBucle;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.QueryParam;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

@Path("overflow") // Cambiamos la ruta a /resources/overflow
public class JakartaEE11Resource {

    private final OverflowControlador controlador = new
OverflowControlador();

    @GET
    @Produces(MediaType.APPLICATION_JSON) // El servidor responderá
automáticamente en formato JSON
    public Response calcularPotencias(@QueryParam("base") int base) {
        try {
            // Validamos que el parámetro se haya enviado correctamente
            if (base < 2) {
                return Response
                    .status(Response.Status.BAD_REQUEST)
                    .entity("{\"error\": \"El número base debe ser
mayor o igual a 2.\"}")
                    .build();
            }

            // Ejecutamos la lógica de control que creamos en el
controlador
            ResultadoBucle resultado =
controlador.procesarBuclePotencias(base);

            // Retornamos el objeto (Jakarta EE lo transforma a JSON
gracias a JSON-B/Jackson)
            return Response
                .ok(resultado)
                .build();

        } catch (Exception e) {
            return Response
                .status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity("{\"error\": \"Error interno en el servidor:
\" + e.getMessage() + \"\"}")
                .build();
        }
    }
}
```

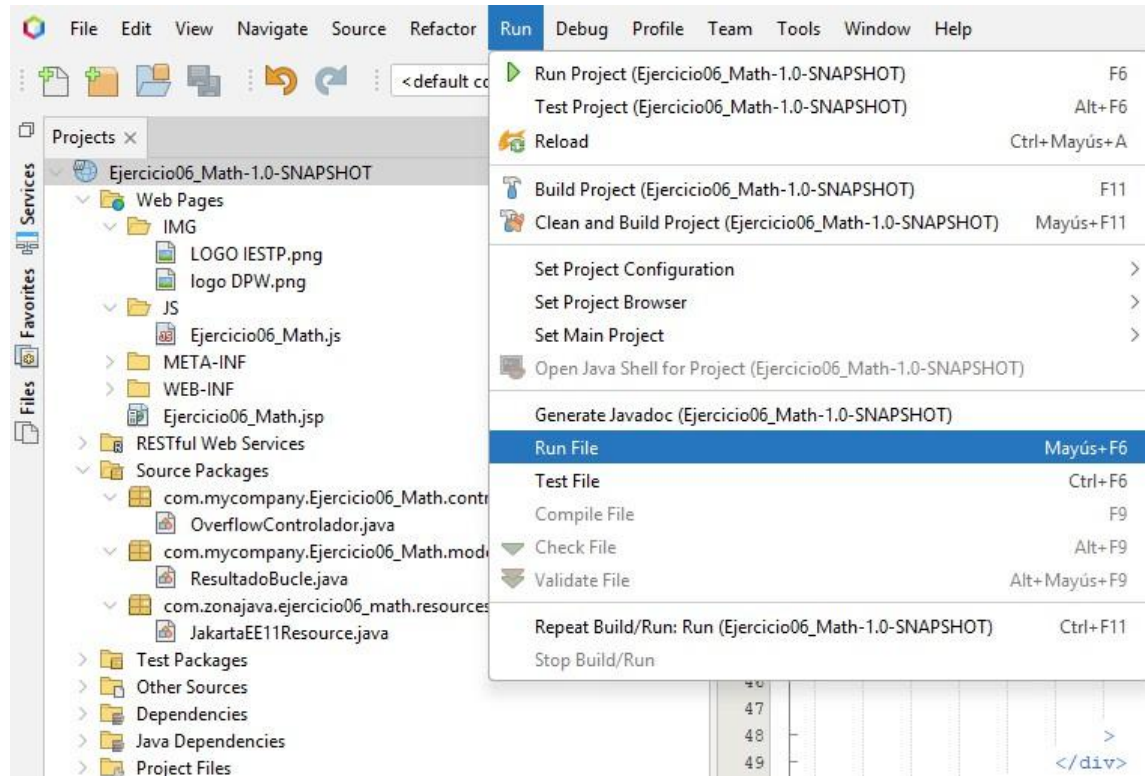
```

        .build();
    }
}
}

```

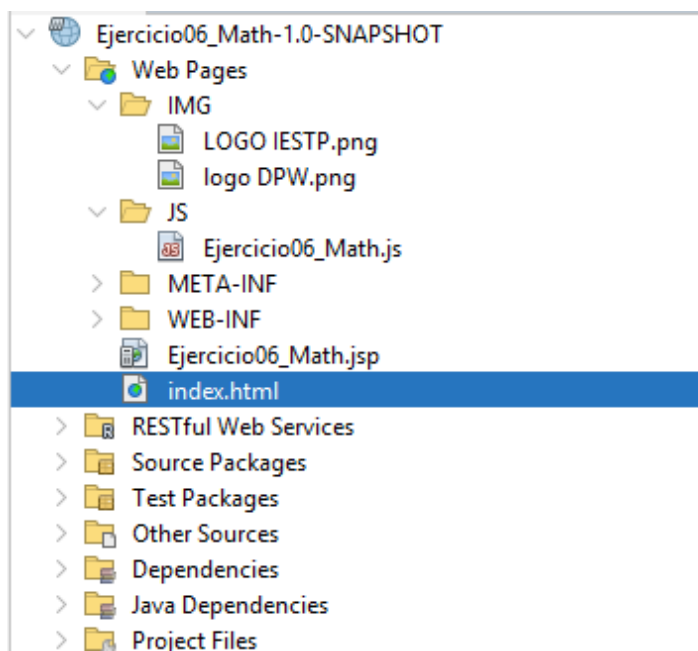
Paso 7: Compilación y Despliegue

1. Ejecuta el proyecto (Run) para ver el Ejercicio06_Math.jsp en tu navegador.



ADICIONAL

1. Puedes eliminar el index.html



VERIFICACION:

